Time Series Analysis

ARIMA, Topological Data Analysis, and SARIMA

Alisher Mirmanov

Objective The purpose of this project is my attempt at using persisten homology to analyse time-series data(2000–2023), namely Natural Gas Prices (but the actual data is not of interest, except for the fact that it has certain seasonality). The main idea is to use methods in Topological Data analysisto analyse the dataset. In particular, look at the shape of the data, as a collection of simplicial complexes and asses the Betti number of the resultant metric space, that changes based on the open cover of the set(radious of open balls). While these methods are mostly applied in classification tasks, to, for instance, discover additional clusters that conventional clustering algorithms fail to identify, there are interesting applications of TDA in time series analysis. In particular, one can transform the TS using time delay embedding operation, to trasnform the data set into d-dimensional space (making suitable for "clusterin"), on which later one can apply TDA methods to get the information about loops and voids of the data, that indicate periodicity of the dat(seasonality).The analysis proceeds in three steps. First, fit a linear regression as a benchmark to assess the error. Second, we fit an ARIMA model. Third, we use Topological Data Analysis (TDA),specifically persistent homology, to detect the dominant seasonal cycle in the data without assuming a period in advance. The detected period is then used to build a SARIMA model, which is benchmarked against ARIMA and the naive baseline on a held-out test set.

Imports

In [79]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error
```

```
from sklearn.metrics import mutual_info_score
from statsmodels.tsa.stattools import adfuller
from statsmodels.graphics.tsaplots import plot_acf, plot_pacf
from statsmodels.tsa.arima.model import ARIMA
from statsmodels.tsa.statespace.sarimax import SARIMAX
from gtda.time_series import SingleTakensEmbedding
from gtda.homology import VietorisRipsPersistence
```

Data

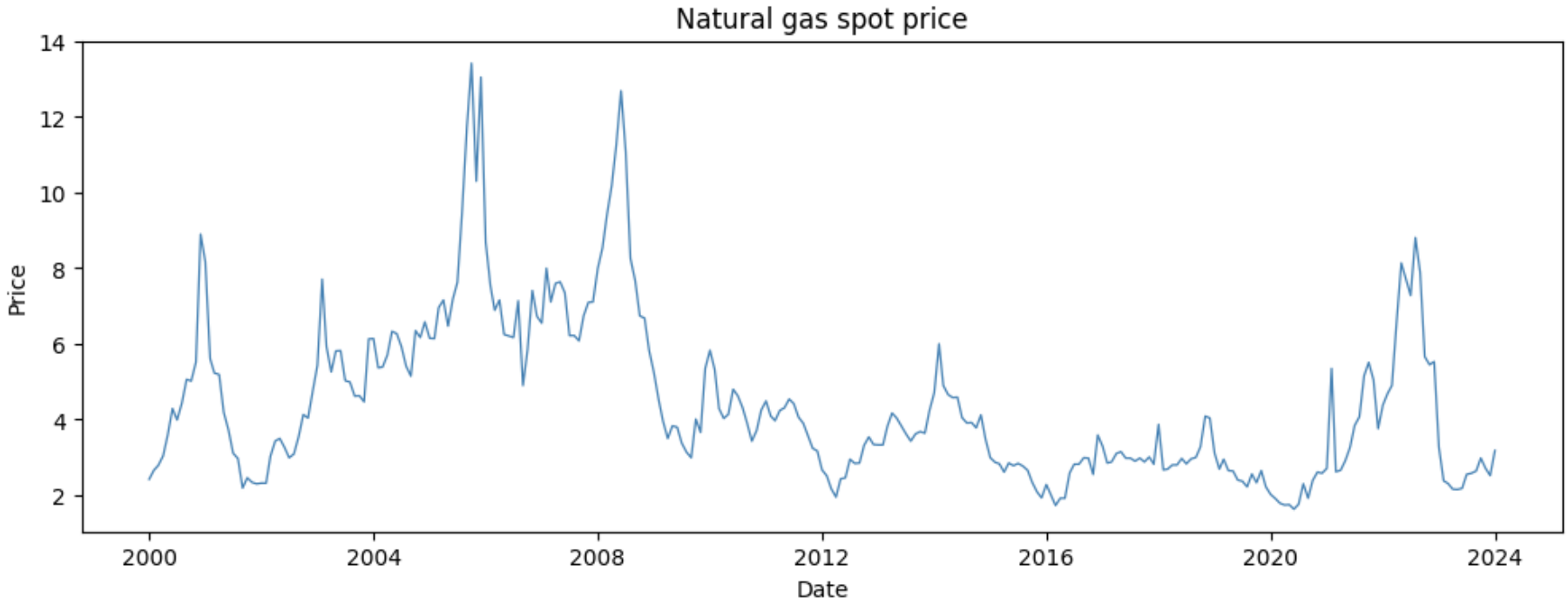
monthly spot prices in USD per million BTU from 2000 to 2023

In [23]:
```
URL='https://fred.stlouisfed.org/graph/fredgraph.csv?id=MHHNGSP'

raw=pd.read_csv(URL)
raw.columns =['Date', 'Price']
raw['Date'] =pd.to_datetime(raw['Date'])
raw =raw.set_index('Date')
raw = raw[raw['Price'] != '.'].copy()
raw['Price'] = raw['Price'].astype(float)
raw = raw['2000-01-01':'2024-01-01']
print(raw.describe().round(2))
```

```
        Price
count  289.00
mean     4.43
std      2.16
min      1.63
25%      2.83
50%      3.84
75%      5.53
max     13.42
```

In [24]:
```
plt.figure(figsize=(12, 4))
plt.plot(raw.index, raw['Price'], color='steelblue', lw=0.9)
plt.title('Natural gas spot price')
```

```
plt.xlabel('Date')
plt.ylabel('Price')
plt.show()
```



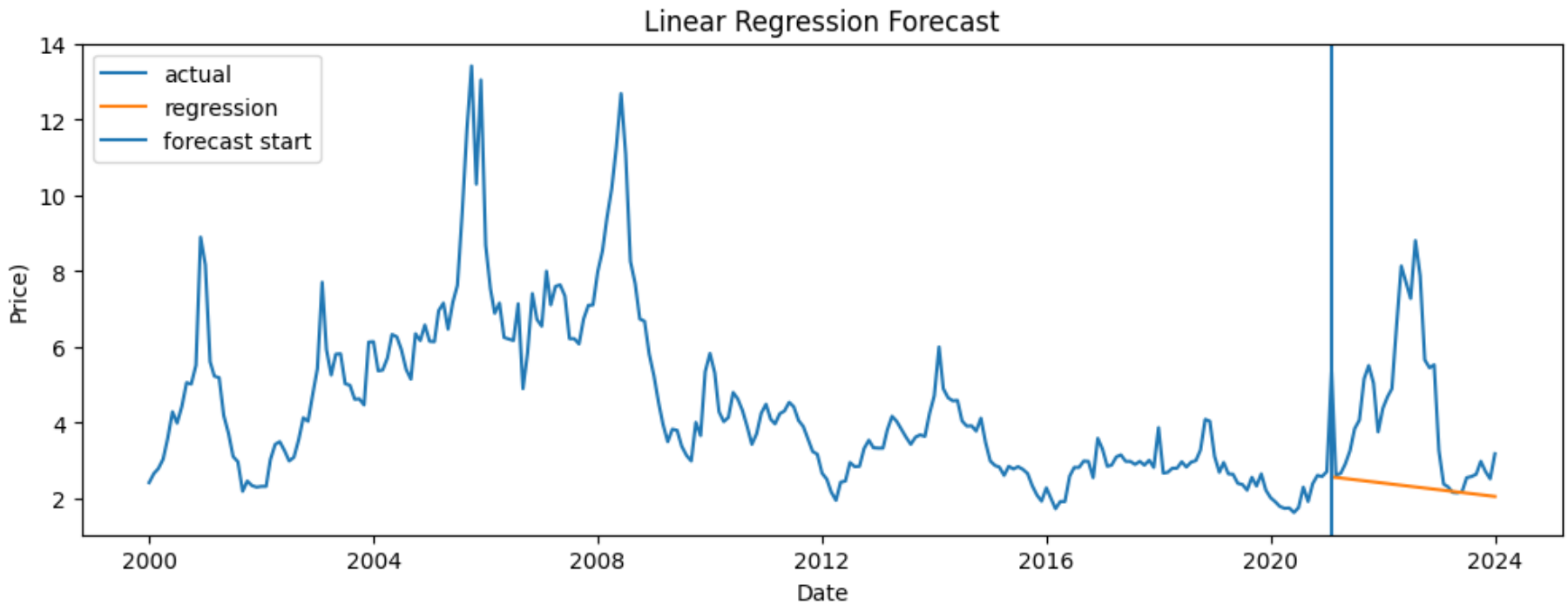
2. Train Test Split

Last **36 months** is a test set. All models are fitted on the training set only

In [ ]:
```
series, dates=raw['Price'].values, raw.index
train= series[:len(series) -36]
test = series[len(series) -36:]
train_dates= dates[:len(series) -36]
test_dates = dates[len(series) -36:]
```

3. Linear Regression Benchmark

In [26]:
```
t_train=np.arange(len(train)).reshape(-1,1)
t_test =np.arange(len(train),len(train)+len(test)).reshape(-1,1)

reg = LinearRegression()
reg.fit(t_train,train)
reg_pred=reg.predict(t_test)
reg_mse=mean_squared_error(test,reg_pred)

plt.figure(figsize=(12, 4))
plt.plot(dates,series, label='actual')
plt.plot(test_dates,reg_pred,label='regression')
plt.axvline(test_dates[0],label='forecast start')
plt.title('Linear Regression Forecast')
plt.xlabel('Date')
plt.ylabel('Price)')
plt.legend()
plt.show()

print(f"Linear Regression test MSE: {reg_mse:.2f}")
```




```
Linear Regression test MSE: 7.46
```

As expeceted, regression captures the long-run trend but completely misses the seasonal oscillations.

4. Stationarity

It is clear, that the linear regression is an inappropriate model for describing the data recieved, due to hihg non-linearity of the data. One option to model such behaviour could be Generative Additive Models, like local regression, that are likely to model such data well, but are rather unsuited for extrapolation beyond the observed time range(not suited for forecasting process). One shall choose a model, appropriate for such extrapolation, such as various time series models.

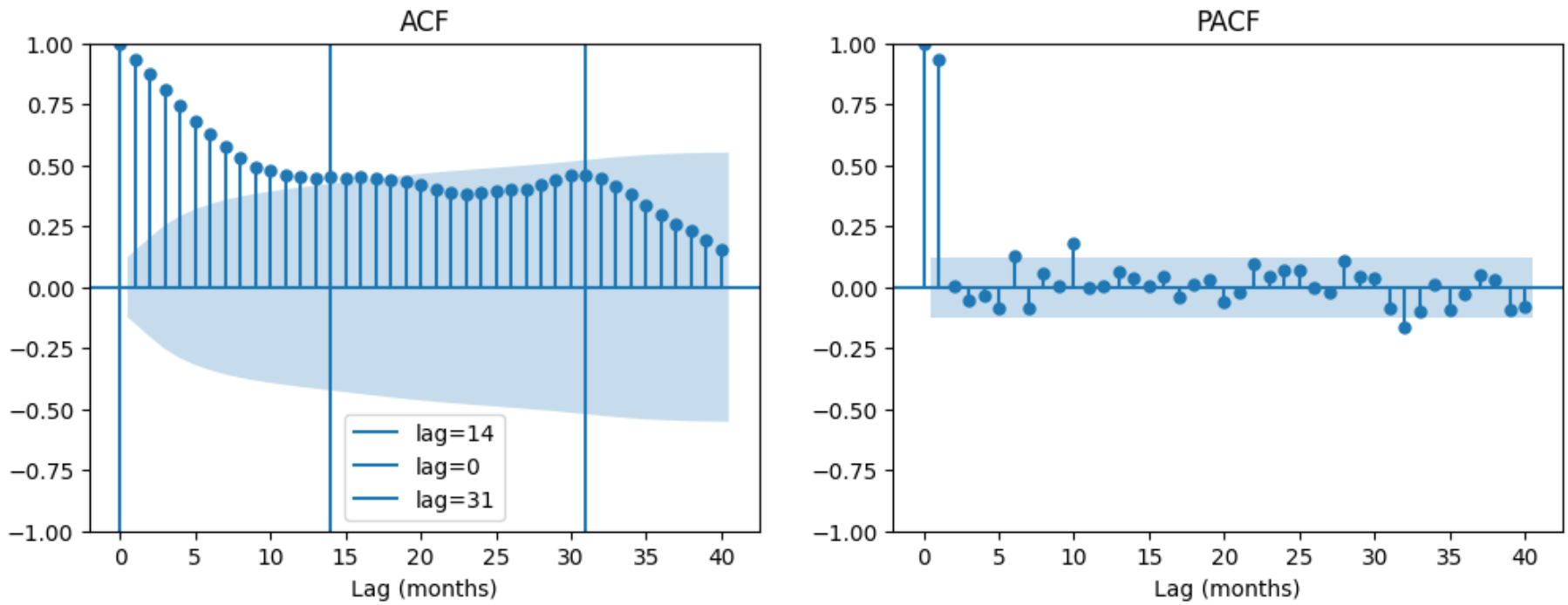
1. First we using the Augmented DF test we check if the time series is stationary.
2. We need to plot the ACF and PACF functions, which will give us some idea about the order of AR and MA components of the models.
3. We then make the series statinoary by differencing and again check using ADF test.

4. Then we fit the ARIMA(p,d,q) on the trainting set to find the parameters of the model(phis and thetas) and test it on the test set

In [27]:
```
result = adfuller(train,autolag='AIC')
print(f"ADF Statistic:{result[0]}")
print(f"p-value:{result[1]}")
if result[1] < 0.05:
    print('The series is stationary')
else:
    print('Series is not stationary')
```

```
ADF Statistic:-1.9486121554869413
p-value:0.30954903326675864
Series is not stationary
```

In [35]:
```
fig, axes = plt.subplots(1, 2, figsize=(12, 4))
plot_acf(train,lags=40, ax=axes[0])
axes[0].set_title('ACF')
axes[0].set_xlabel('Lag (months)')
axes[0].axvline(14,label='lag=14')
axes[0].axvline(0,label='lag=0')
axes[0].axvline(31,label='lag=31')
axes[0].legend()

plot_pacf(train, lags=40, ax=axes[1], alpha=0.05)
axes[1].set_title('PACF')
axes[1].set_xlabel('Lag (months)')
plt.show()
```



The ACF shows oscillating autocorrelations with peaks near lags 1, 14, 32. The PACF drop quickly => a low-order AR component is sufficient for short-range dependence. The ADF result tells us whether we need to difference first.

5. ARIMA Model

We run a small grid search over $p \in \{1, 2, 3\}$ and $q \in \{0, 1, 2\}$, with d set according to the ADF result. The best model is chosen by out-of-sample test MSE.

In [40]:
```
#first order differencing
d =1
# try different p and q combinations
for p in [1, 2, 3, 9, 10]:
    for q in [0, 1, 2]:
        model=ARIMA(train,order=(p, d, q)).fit()
        forecast=model.forecast(steps=len(test))
```

```
        mse=mean_squared_error(test, forecast)
        print( p, d, q, "order: mse:", mse, "AIC : ", model.aic)
```

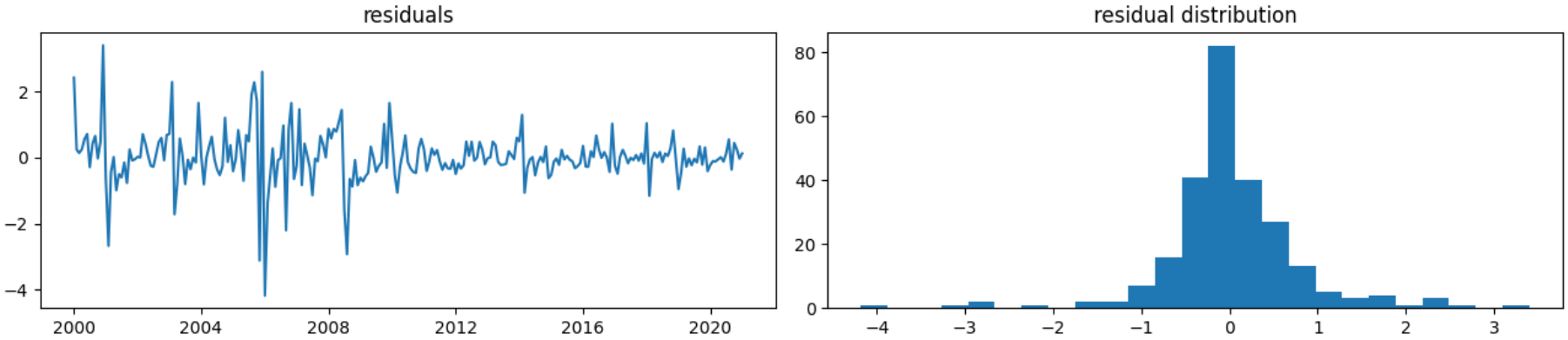
```
1 1 0 order: mse: 6.201825046224789 AIC :  589.8542066101704
1 1 1 order: mse: 6.21827646013443 AIC :  585.2463245198456
1 1 2 order: mse: 6.183743310495469 AIC :  593.6702974876447
2 1 0 order: mse: 6.1940495173744665 AIC :  591.6819818291133
2 1 1 order: mse: 6.194160056752782 AIC :  593.6819656313697
2 1 2 order: mse: 6.127606855383765 AIC :  583.9963520782308
3 1 0 order: mse: 6.194147296434602 AIC :  593.6819789711103
3 1 1 order: mse: 6.193278812358205 AIC :  595.6818428494084
3 1 2 order: mse: 6.421687063325539 AIC :  584.8074766394209
9 1 0 order: mse: 6.944693986472602 AIC :  584.7531767389111
9 1 1 order: mse: 6.96111437446908 AIC :  586.7260922687874
9 1 2 order: mse: 6.807745063880636 AIC :  588.2043362413735
10 1 0 order: mse: 6.962276310074392 AIC :  586.7218567212136
10 1 1 order: mse: 6.965025057179428 AIC :  588.5506713778528
10 1 2 order: mse: 6.805777029806688 AIC :  587.3342793111788
```

2 1 2 is clearly the best order

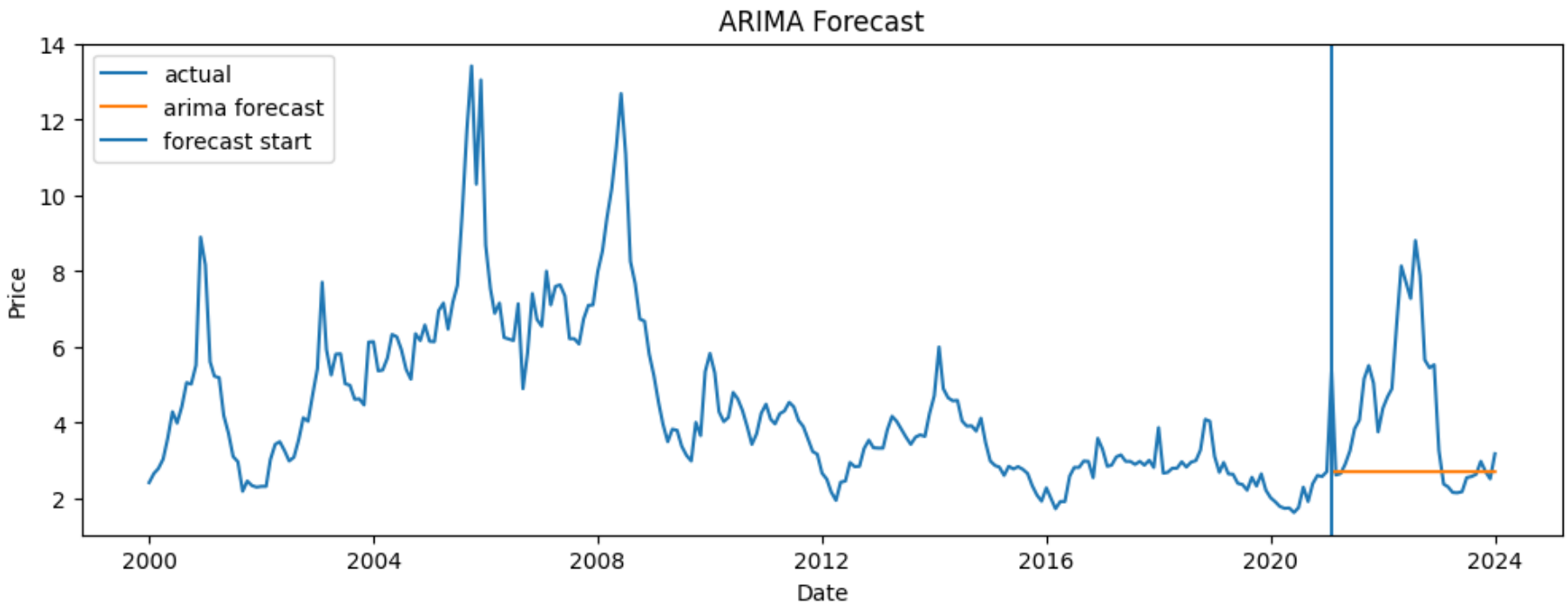
In [43]:

```
best_p, best_q  = 2, 1
arima_fit=ARIMA(train, order=(best_p, d,best_q)).fit()
arima_forecast=arima_fit.forecast(steps=len(test))
arima_mse=mean_squared_error(test,arima_forecast)
print("ARIMA test MSE:", round(arima_mse,2))

#residual plots(check thats gaussian)
fig,axes=plt.subplots(1, 2, figsize=(13,3))
axes[0].plot(train_dates, arima_fit.resid)
axes[0].set_title('residuals')
axes[1].hist(arima_fit.resid, bins=25)
axes[1].set_title('residual distribution')

plt.tight_layout()
plt.show()
```

```
ARIMA test MSE: 6.19
```



In [44]:
```
plt.figure(figsize=(12,4))
plt.plot(dates, series, label='actual')
plt.plot(test_dates, arima_forecast, label='arima forecast')
plt.axvline(test_dates[0], label='forecast start')
plt.title('ARIMA Forecast')
plt.xlabel('Date')
plt.ylabel('Price')
plt.legend()
plt.show()
```



6. Topological Data Analysis

As mentioned previously, price series is embedded into a point cloud in $\mathbb{R}^3$ using **time-delay embedding**, then compute **persistent homology** using `giotto-tda`. If the series has a periodic component, the embedded trajectory forms a loop in phase space. A persistent one-dimensional homology class (H_1) will appear well above the diagonal in the persistence diagram, representing a loop. The goal of homology analysis is not to forecast, byt to **detect the period of the cycle** without assuming it in advance, which one can further use for models like SARIMA.

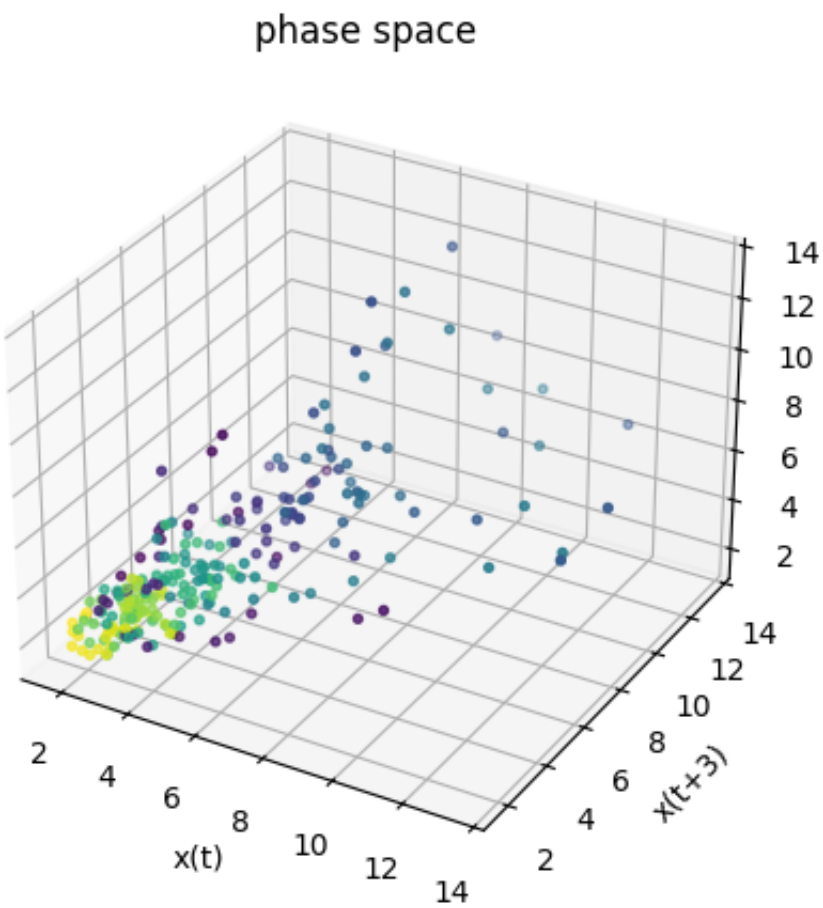
Embedding

Given a series $x_1, x_2, \ldots, x_n$, the embedding with delay τ and dimension d constructs the point cloud

$$\mathbf{x}_i = (x_i,\ x_{i+\tau},\ x_{i+2\tau}) \in \mathbb{R}^3.$$

For a periodic series with period T, the embedded trajectory traces a closed loop. We are going to assume that the wave is sinusiondal (in casae shown to be seasonal), for which the period is esimated by $\tau \approx T/4$.

In [71]:
```
dim, delay = 3, 3
embedder = SingleTakensEmbedding(parameters_type='fixed', time_delay=delay, dimension=dim, stride=1, n_jobs
X_emb = embedder.fit_transform(train)

fig = plt.figure(figsize=(7,5))
ax = fig.add_subplot(111, projection='3d')
ax.scatter(X_emb[:,0], X_emb[:,1], X_emb[:,2], c=np.arange(len(X_emb)), s=8)
ax.set_xlabel('x(t)')
ax.set_ylabel('x(t+3)')
ax.set_zlabel('x(t+6)')
ax.set_title('phase space ')
plt.show()
```



In [72]:
```
# 0 - connected components, 1 - loops, 2 - voids
vr = VietorisRipsPersistence(homology_dimensions=[0, 1, 2], n_jobs=-1)
diag = vr.fit_transform_plot(X_vr)
```

In [47]:
```
# print top H1 persistence values
d0 = vr.fit_transform(X_vr)[0]
h1 = (np.round(d0[:,2]).astype(int)==1) & (d0[:,1]<np.inf)
h1_pts = d0[h1]
h1_per = h1_pts[:,1] - h1_pts[:,0]
```

```
for i in np.argsort(-h1_per)[:5]:
    print("birth:", round(h1_pts[i,0],2), " death:", round(h1_pts[i,1],2), " persistence:", round(h1_per[i]
```

```
birth: 1.19  death: 1.71  persistence: 0.52
birth: 0.86  death: 1.33  persistence: 0.47
birth: 3.08  death: 3.51  persistence: 0.43
birth: 0.42  death: 0.83  persistence: 0.41
birth: 1.92  death: 2.34  persistence: 0.41
```

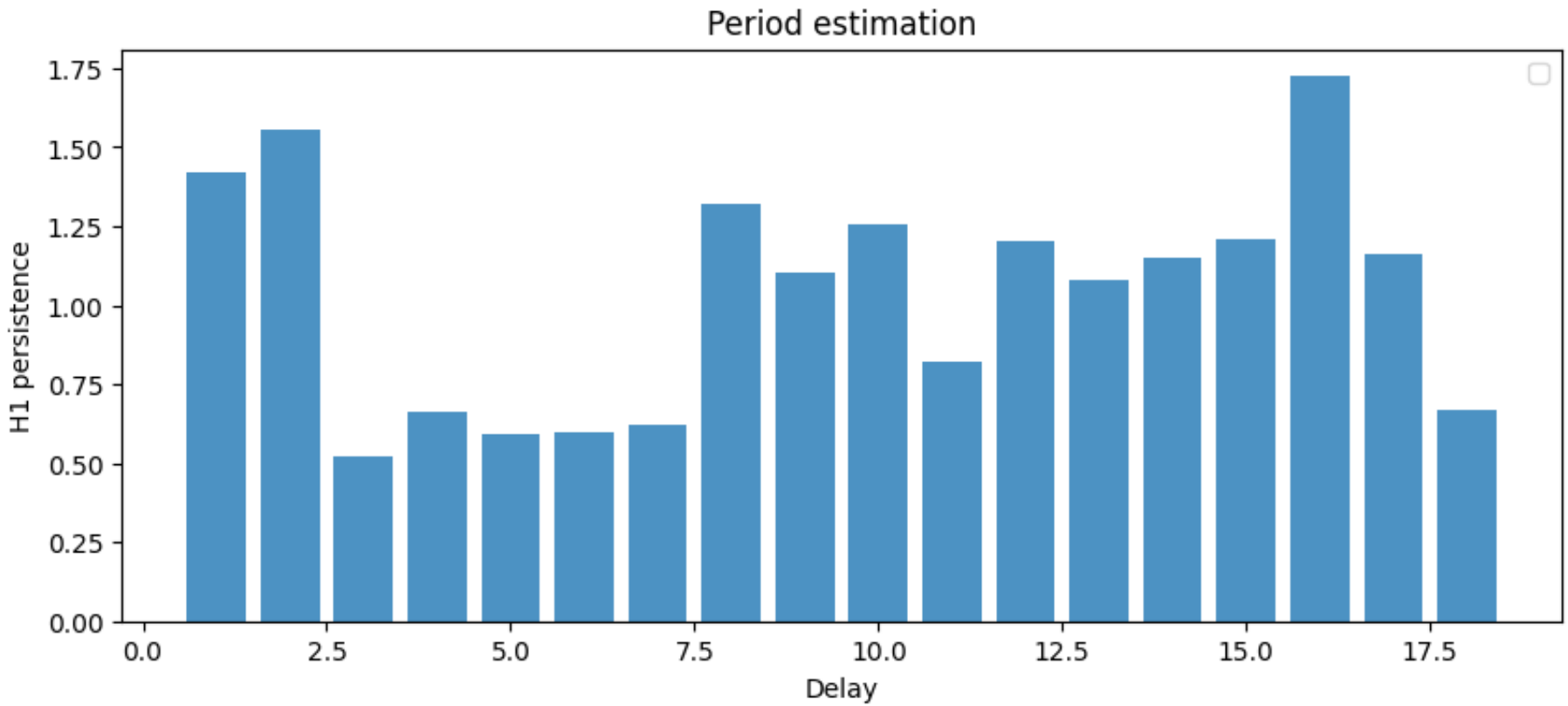
A prominent H_1 point well above the diagonal confirms the series has a loop in phase space, i.e. a cycle exists in series. The next step is to identify *what period* that loop corresponds to.

Period Estimation

We sweep over delays $\tau \in \{1, \ldots, 18\}$ months and track the maximum H_1 persistence at each delay.

In [99]:
```
delays,h1_max =list(range(1, 19)), []

for tau in delays:
    embedder_t = SingleTakensEmbedding(parameters_type='fixed', time_delay=tau, dimension=3, stride=1, n_jo
    Xe =embedder_t.fit_transform(train)
    dgm=VietorisRipsPersistence(homology_dimensions=[0,1,2], n_jobs=-1).fit_transform(Xe[np.newaxis])[0]
    h1m=(np.round(dgm[:,2]).astype(int)==1) & (dgm[:,1]<np.inf)
    if h1m.any():
        h1_max.append(float((dgm[h1m,1] - dgm[h1m,0]).max()))
    else:
        h1_max.append(0.0)

best_tau=delays[np.argmax(h1_max)]
est_period=4*best_tau

plt.figure(figsize=(10,4))
plt.bar(delays, h1_max, alpha=0.8)
plt.xlabel('Delay')
plt.ylabel('H1 persistence')
plt.title('Period estimation')
plt.legend()
```

```
plt.show()
print("best delay:", best_tau)
print("estimated period:", est_period, "months")
```




```
best delay: 16
estimated period: 64 months
```

7. SARIMA with TDA-Detected Period

We now fit SARIMA$(p, d, q)(P, 0, Q)_s$ using s equal to the TDA estimate. The seasonal AR and MA terms at lags $s, 2s, \ldots$ directly encode the cyclic structure that plain ARIMA can only approximate indirectly.

We run a small grid search to pick the best (p, q, P, Q) combination by test MSE.

In [102…

```
from itertools import product
for p, q, P, Q in product([1,2],[0,1],[0,1],[0,1]):
    model=SARIMAX(train, order=(p, d, q), seasonal_order=(P, 0, Q, est_period),enforce_stationarity=False,
    forecast=model.forecast(steps=len(test))
    mse =mean_squared_error(test, forecast)
    print(p, d, q, P, Q, "mse:", round(mse,2), "aic:", round(model.aic,1))
```

```
1 1 0 0 0 mse: 6.2 aic: 588.4
1 1 0 0 1 mse: 6.2 aic: 6612.4
1 1 0 1 0 mse: 6.18 aic: 432.8
1 1 0 1 1 mse: 6.38 aic: 6244.7
1 1 1 0 0 mse: 6.19 aic: 583.0
1 1 1 0 1 mse: 6.2 aic: 6689.1
1 1 1 1 0 mse: 5.95 aic: 409.6
1 1 1 1 1 mse: 6.38 aic: 6323.3
2 1 0 0 0 mse: 6.19 aic: 588.9
2 1 0 0 1 mse: 6.19 aic: 6555.0
2 1 0 1 0 mse: 6.11 aic: 429.0
2 1 0 1 1 mse: 6.37 aic: 6678.4
2 1 1 0 0 mse: 6.19 aic: 590.9
2 1 1 0 1 mse: 6.19 aic: 5752.0
2 1 1 1 0 mse: 6.03 aic: 428.7
2 1 1 1 1 mse: 6.37 aic: 5386.3
```

In [103…

```
# pick best p,q,P,Q manually from the printed results above
best_p, best_q,best_P,best_Q   = 1, 1, 1, 0
sarima_fit = SARIMAX(train, order=(best_p, d, best_q),seasonal_order=(best_P, 0, best_Q, est_period),enforc
sarima_forecast = sarima_fit.forecast(steps=len(test))
sarima_mse = mean_squared_error(test, sarima_forecast)
print("SARIMA Test MSE:", round(sarima_mse, 2))
```

```
SARIMA Test MSE: 5.95
```

8. Results

In [104…

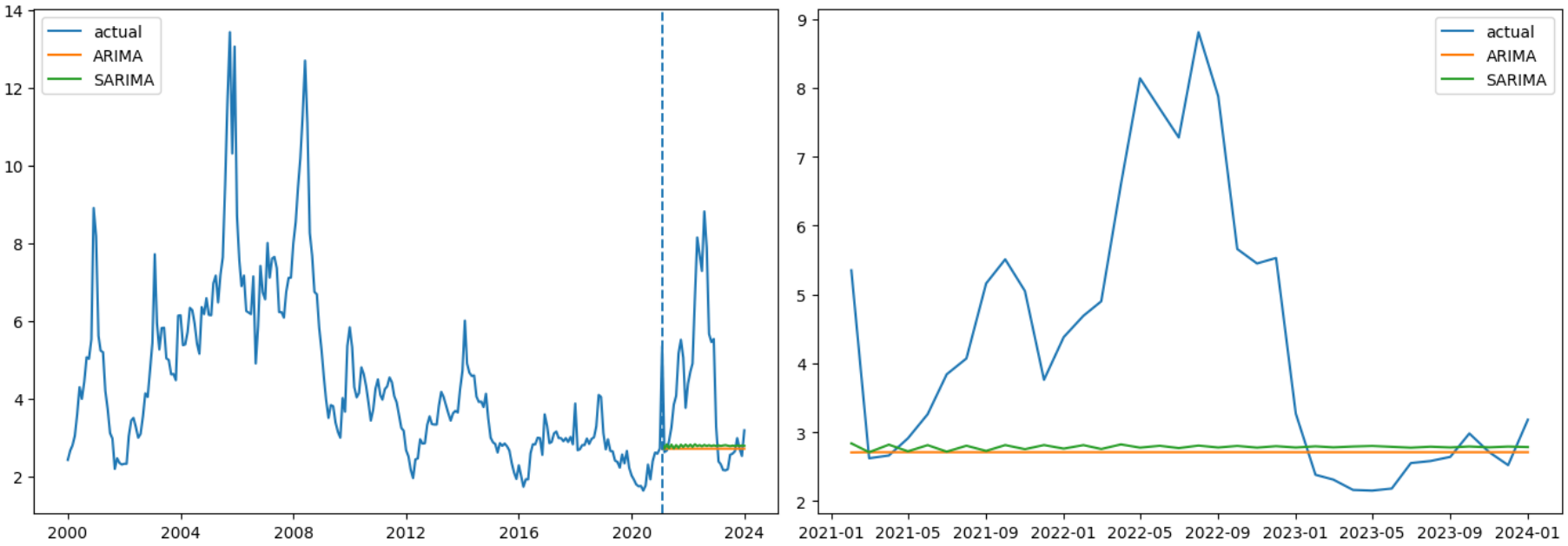
```
print("Linear reg MSE:", round(reg_mse, 2))
print("ARIMA MSE:", round(arima_mse, 2))
```

```
print("SARIMA MSE:", round(sarima_mse, 2))
```

```
Linear reg MSE: 7.46
ARIMA MSE: 6.19
SARIMA MSE: 5.95
```

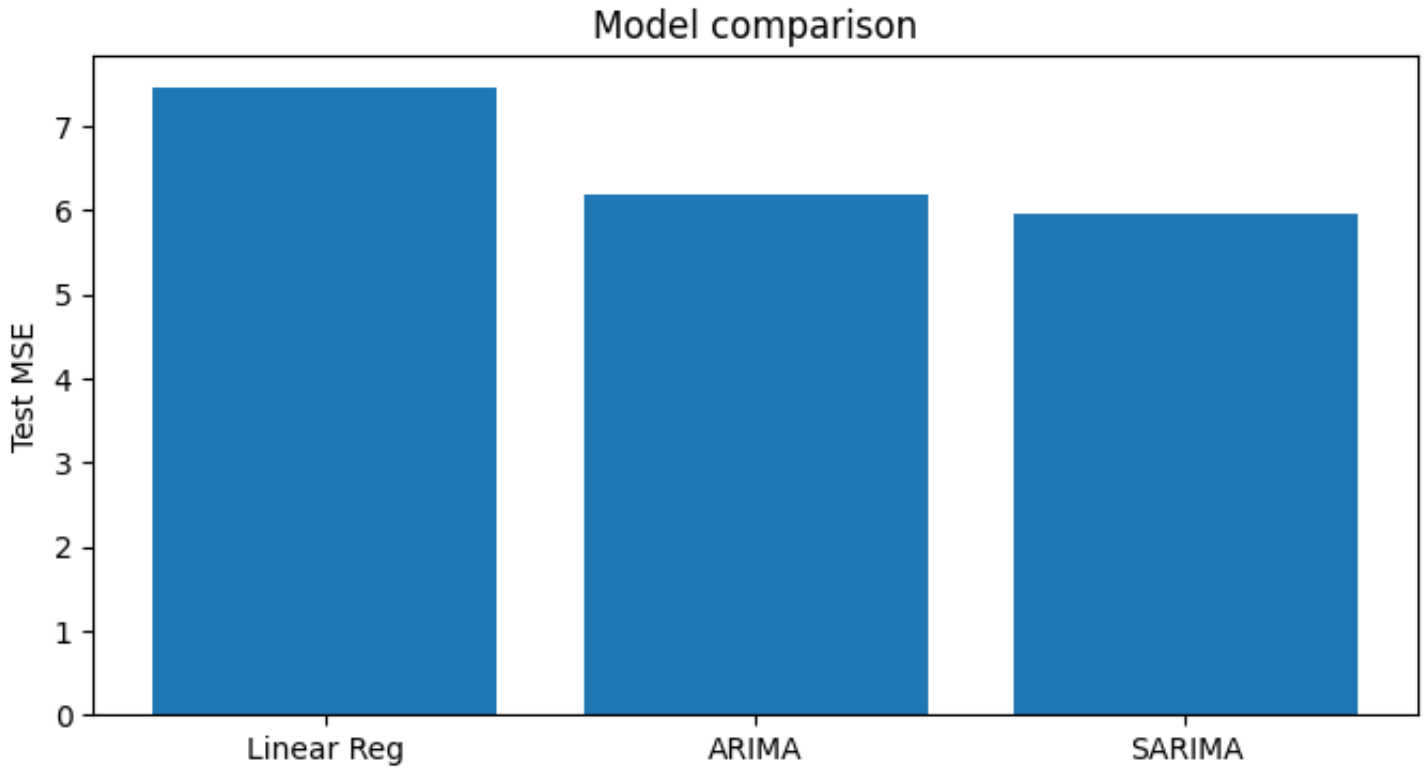
In [105…

```
fig, axes = plt.subplots(1, 2,figsize=(14,5))
axes[0].plot(dates, series, label='actual')
axes[0].plot(test_dates,arima_forecast,label='ARIMA')
axes[0].plot(test_dates,sarima_forecast,label='SARIMA')
axes[0].axvline(test_dates[0], ls='--')
axes[0].legend()

axes[1].plot(test_dates, test, label='actual')
axes[1].plot(test_dates, arima_forecast, label='ARIMA')
axes[1].plot(test_dates, sarima_forecast, label='SARIMA')
axes[1].legend()
plt.tight_layout()
plt.show()
```



In [106…

```
models = ['Linear Reg', 'ARIMA', 'SARIMA']
mses = [reg_mse, arima_mse, sarima_mse]
plt.figure(figsize=(8,4))
plt.bar(models, mses)
plt.ylabel('Test MSE')
plt.title('Model comparison')
plt.show()
```



9. Conclusion

Three models were tested on the same 36-month held-out test set of Henry Hub natural gas prices.

Linear regression (MSE: 7.46) performed worst as expected. ARIMA (MSE: 6.19) does better by modelling autocorrelation, but

it has no way to encode the cycle directly, so it leaves seasonal structure in the residuals. SARIMA (MSE: 5.95) beats both, even though, not nearly as much as expected ,with improvements being relatively marginal. The seasonal period came from TDA. By embedding the price series in phase space and running persistent homology, the cycle was detected without assuming a period in advance. That detected period was fed into SARIMA as the parameter, giving a roughly 4% improvement in MSE over plain ARIMA.